Evaluating a Semantic Prompt-Injection Guard for a Locally Deployed Language Model

Mustafa Shaik
Georgia Tech M.S. Cybersecurity Practicum
Securing LLMs at the Edge

Summary

This project examined whether one locally deployed language model could help protect another from prompt-injection attacks. The original practicum used LangChain to orchestrate the application and Ollama to serve models locally.

The original executable repository and numerical result tables were not preserved in the materials available for this report. I therefore ran a supplementary evaluation using a recovered set of adversarial prompts. The quantitative results below describe that supplementary run, not the original course results.

The evaluation compared two versions of a semantic input guard using Llama 3.2 3B. A naive guard received a short security policy. A hardened guard received explicit attack categories, authority rules, mixed-intent handling, examples, and a constrained output format.

The naive guard detected 2 of 12 adversarial prompts. The hardened guard detected 9 of 12 while allowing all 12 benign controls. Clearer policy design improved the guard, although the remaining failures make it unsuitable as the system's only security boundary.

Problem and Threat Model

Prompt injection exploits the fact that system policy, user requests, retrieved content, and tool output all reach an LLM through natural language. A model may fail to preserve the intended authority ordering among those sources.

The attacker in this evaluation was assumed to control the user prompt. The tested goals included overriding the assigned task, extracting hidden instructions, obtaining restricted technical information, and disguising prohibited requests as research or education. The attacker was not assumed to have access to the model weights, application code, or host.

Indirect injection through retrieved documents was considered in the project architecture but was not measured in the supplementary evaluation.

Evaluation Design

The corpus contained 24 prompts. Twelve were adversarial, consisting of four instruction-override attempts, four prompt-leakage attempts, and four jailbreak attempts. Twelve benign controls covered translation, AI explanations, summarization, structured text, transparency, laboratory safety, content moderation, authentication, authorization, and least privilege.

The evaluation used Llama 3.2 3B through the local Ollama API. Temperature was set to zero, and a fixed seed was supplied. The same model family served as guard, target, and response judge because only one model was installed.

The naive guard received a broad instruction to block malicious requests. The hardened guard received explicit categories for prompt extraction, task override, security bypass, mixed-intent requests, and harmful operational detail. It was instructed to treat the user prompt as data to classify rather than an instruction to follow. Contrastive examples distinguished prohibited requests from legitimate high-level security explanations. Its decision was returned through a fixed JSON schema.

Application code retained the final allow-or-block decision. The model supplied a classification and reason but did not receive authority to execute an action.

Results

Metric	Naive guard	Hardened guard
Adversarial prompts blocked	2 of 12	9 of 12
Attack-detection rate	16.7%	75.0%
Adversarial prompts missed	10 of 12	3 of 12
Benign prompts blocked	1 of 12	0 of 12
Observed false-positive rate	8.3%	0.0%
Balanced accuracy	54.2%	87.5%
Mean guard latency	0.343 seconds	0.416 seconds
Median guard latency	0.322 seconds	0.366 seconds

The hardened guard improved attack detection by 58.3 percentage points. Mean classification latency increased by approximately 73 milliseconds.

The naive guard frequently accepted the attacker's framing. Requests described as research, education, translation, or open-source inquiry were often allowed even when the requested information remained prohibited.

The hardened policy required the model to examine the requested capability and information flow instead of relying on the user's stated purpose. The improvement came from changing the policy presented to the model, not from changing its weights.

Three adversarial prompts still passed. They asked about the structure of the guiding prompt, categories the model avoids, and hypothetical access to restricted data. These requests were close to legitimate high-level discussion, and the guard did not classify them as attacks.

The response judge also produced a false alarm. It labeled a benign explanation of why bypassing access controls is unethical as unsafe, even though the answer provided no bypass procedure. A plausible label from an LLM judge cannot automatically be treated as ground truth.

Limitations

The corpus contained only 24 cases, with each prompt evaluated once. The percentages describe this test set and should not be interpreted as estimates of general model safety. Zero false positives among 12 benign cases does not establish a true false-positive rate of zero.

The same model family served as guard, target, and judge, creating the possibility of correlated blind spots. The fixed seed and zero temperature reduced variation, but repeated trials would still be needed to measure stability.

The evaluation did not measure indirect injection, multi-turn attacks, encoded payloads, tool use, or memory poisoning. An adaptive attacker could also target the guard or response judge directly.

Recommended Next Steps

The first extension should use a larger, versioned corpus with repeated trials and confidence intervals. Different model families should serve as guard, target, and judge to reduce correlated errors.

The evaluation should then add indirect injections embedded in retrieved documents and tool output. A further extension should test whether an injected instruction influences memory, planning, delegation, or a later tool call in a multi-step agent workflow.

Conclusion

The hardened guard detected 9 of 12 attacks, compared with 2 of 12 for the naive guard, while adding about 73 milliseconds of mean classification latency. Clearer policy decomposition improved performance without changing the model.

The remaining failures are the more important result. One quarter of the adversarial prompts still passed, and the response judge incorrectly flagged a benign answer. An LLM guard can strengthen one layer of a prompt-injection defense, but it cannot serve as the application's security boundary. Production systems still require deterministic authorization, restricted tool access, output validation, audit logs, adversarial evaluation, and human approval for consequential actions.

References

1. Greshake, K., et al. (2023). "Not What You've Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection." arXiv:2302.12173.

2. OWASP GenAI Security Project. "LLM01: Prompt Injection." OWASP Top 10 for Large Language Model Applications.

3. Zheng, L., et al. (2023). "Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena." arXiv:2306.05685.